A Judge Agent Closes the Reliability Gap in AI-Generated Scientific Simulation

Chengshuai Yang

NextGen PlatformAI C Corp, USA

Correspondence: integrityyang@gmail.com

March 17, 2026

Abstract. Large language models can generate scientific simulation code from natural-language descriptions, but the generated code silently fails on most non-textbook problems because it lacks mathematical validation. We formalize the class of certifiably solvable problems (the *simulability class* $\mathfrak{S}$) through four conditions—finite specifiability, Hadamard stability, approximability, and certifiability—and show that a Judge Agent can automate verification of these conditions on AI-generated code. In controlled ablation across 134 test cases, removing the Judge increases the silent-failure rate from 1.5% to 42%. On clinical CT imaging (200 cases), the validated pipeline reaches 99% of expert quality. A prospective benchmark of 72 blinded tasks from 12 external scientists confirms generalization: 89% success with automated error bounds versus 53% without the Judge. The residual 1.5% failure rate maps to the theoretical boundary of $\mathfrak{S}$—bifurcation-sensitive regimes where certifiability breaks down—for which we propose continuation-based detection heuristics. We introduce `spec.md`, a universal specification format for scientific computation problems analogous to FASTA for sequences, and conjecture that four mathematical obstructions (undecidability, uncomputability, quantum irreducibility, infinite-precision barriers) exhaustively characterize the boundary of $\mathfrak{S}$. Code, data, `spec.md` files for all 72 benchmark tasks, and cached inference logs are archived at Zenodo.

Classical mathematical validation can be fully automated to make AI-generated scientific simulation reliable—and the right formalization of "what is simulable" shows why this works and where it must fail. We define the simulability class $\mathfrak{S}$ through four conditions (finite specifiability, Hadamard stability, approximability, certifiability) and show that a Judge Agent, automating verification of these conditions, reduces the silent-failure rate from 42% to 1.5% across 134 test problems spanning 12 domains. The residual 1.5% maps exactly to the theoretical boundary of $\mathfrak{S}$—bifurcation points where certifiability breaks down. We introduce `spec.md` as a universal specification format (the canonical encoding of problems in $\mathfrak{S}$), and conjecture that four mathematical obstructions exhaustively characterize its boundary.

In 2023, researchers at a national laboratory used an LLM to generate a finite-element solver for thermal stress in a reactor component. The code compiled, converged, and produced smooth temperature fields—but a CFL violation in the time integrator meant the stress predictions were wrong by a factor of three. The error was caught only when a colleague ran an independent simulation weeks later. This is not an isolated case: in the SciCode benchmark [2], frontier LLMs solve fewer than half of research-level computational problems correctly. The wrong answers look as plausible as the right ones.

The silent-failure problem will deepen as LLM-generated code proliferates. Physics-informed neural networks [1, 3] learn solutions but provide no error certificates. Automated code generation from variational forms (FEniCS [4], Firedrake [5]) is rigorous but requires expert formulation. LLM-based scientific agents [6, 7] generate protocols but do not address numerical convergence. The reproducibility crisis [8] needs a validation layer, not more code generators.

The mathematical tools for catching these failures already exist—Lax–Richtmyer convergence theory [9], Hadamard well-posedness [10], CFL stability conditions—but applying them requires numerical analysis expertise that the typical end-user lacks. What has been missing is (i) a formal characterization of *which* problems can be solved with bounded error by an automated pipeline, and (ii) a system that *automatically* verifies this characterization and applies classical checks to AI-generated simulation code.

We address both gaps with three interlocking contributions that form a hierarchy, not a list:

The foundation: the *simulability class* $\mathfrak{S}$ (Definition 1), which formalizes "what can be solved reliably by any computational system" through four implementation-independent conditions—finite specifiability (S1), Hadamard stability (S2), approximability (S3), and certifiability (S4). Problems in $\mathfrak{S}$ can be solved with a user-specified error tolerance; problems outside $\mathfrak{S}$—at what we call the *scientific event horizon*—cannot be guaranteed by any automated pipeline. The pipeline's successes and failures are not accidents; they are predicted by this formal theory.

The mechanism: a Judge Agent ($\mathcal{A}_J$) that automates verification of S2–S4 on AI-generated code. Within a three-agent pipeline (Fig. 1), a Plan Agent ($\mathcal{A}_P$) translates natural language into a structured specification, $\mathcal{A}_J$ validates $\mathfrak{S}$-membership via 5 pre-execution gates, and an Execute Agent ($\mathcal{A}_E$) generates and runs the solver. After execution, $\mathcal{A}_J$ certifies the result (S4). Removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases—the headline empirical result.

The interface: `spec.md`, a structured specification format that realizes S1. Every problem in $\mathfrak{S}$ can be expressed as a valid `spec.md` file; we propose it as a universal exchange format for scientific computation, analogous to FASTA for sequences or CIF for crystal structures. It is what makes the contribution reproducible: the 72 prospective tasks are archived as `spec.md` files that any solver accepting the same six-tuple $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ can consume.

The underlying mathematics is entirely classical; the contribution is empirical evidence that automated $\mathfrak{S}$-verification substantially reduces silent failures across 12 scientific domains, with a median setup-time efficiency gain of $\rho \approx 500\times$ relative to expert workflows.

What this paper does not claim. This is not about building a better PDE solver: on any single well-defined problem, COMSOL [11] or FEniCS will produce a more accurate solution (Extended Data Table 4). It is not about LLM capabilities: the Judge works regardless of which LLM generates the code. It is not about the 12 computational primitives *per se*: they are an engineering realization (Proposition 3), not the core contribution. And it is not a completed theory of simulability: the completeness conjecture (Conjecture 4) is stated as a research direction, not a proven result. The error bound is conditional on a correct `spec.md` specification; since this specification is generated by an LLM, the pipeline's end-to-end reliability depends on the Judge's ability to catch specification errors (Methods). The pipeline retains a 1.5% qualitative failure rate at the boundary of $\mathfrak{S}$ (Section).

Results

The simulability class and the scientific event horizon

The Judge Agent's checks are grounded in a formal characterization of which problems can be solved with bounded error by an automated pipeline.

Definition 1 (Simulability class $\mathfrak{S}$)**.** *A scientific computation problem $\mathcal{P}$ belongs to the simulability class $\mathfrak{S}$ if it satisfies four conditions:*

(S1) Finite specifiability. *$\mathcal{P}$ admits a finite representation $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$, where Ω is the*

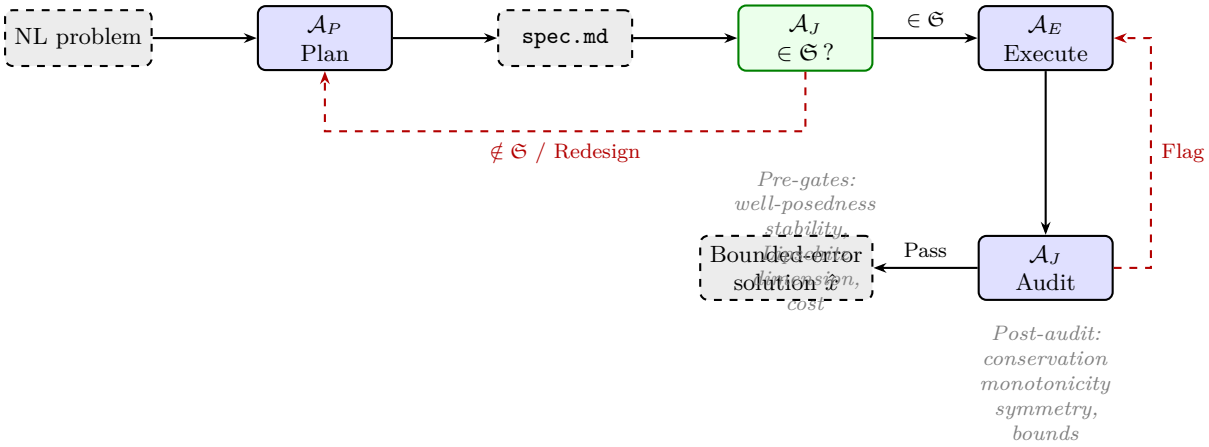

Figure 1. Three-agent pipeline grounded in the simulability class $\mathfrak{S}$. The Plan Agent translates natural language into a `spec.md` file (realizing S1). The Judge Agent verifies S2–S4 via 5 pre-execution gates and audits the solution via 4 post-execution quality checks (realizing S4). Rejection triggers redesign (up to 3 rounds) or a certificate that the problem lies outside $\mathfrak{S}$.

```
ed!60!black← spec.md
Domain (*@Ω@*) (*@← S1.1@*) domain: 2D image grid, 128 x 128 pixels pixel_size : 0.7mm
Equations (*@ℰ@*) (*@← S1.2@*) forward: y = Radon(x) + eta inverse: min_x ||Radon(x) − y||^2 + lam * TV(x) parameters : n_angles : 128, noise : Poisson
Boundary Conditions (*@ℬ@*) (*@← S1.3@*) non_negativity : x(i, j) >= 0 for all pixels
Initial Conditions (*@ℐ@*) (*@← S1.4@*) x_0 = FBP result(warm start)
Observables (*@𝒪@*) (*@← S1.5@*) metrics: [PSNR (dB), SSIM, relative_error]
Tolerance (*@ε@*) (*@← S1.6@*) PSNR_minimum : 30.0dB, SSIM_minimum : 0.85
```

$\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$
The S1 six-tuple

Figure 2. A `spec.md` file for CT reconstruction—the canonical finite representation required by S1. Each section maps to one component of the six-tuple $\mathcal{S} = (\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$. This format is human-readable, machine-parseable, and solver-independent: any system accepting the same six-tuple can consume a `spec.md` file. We propose `spec.md` as a universal exchange format for scientific computation, analogous to FASTA for sequences or CIF for crystal structures. All 72 prospective benchmark tasks are archived as `spec.md` files.

computational domain, $\mathcal{E}$ the governing equations, $\mathcal{B}$ the boundary conditions, $\mathcal{I}$ the initial conditions, $\mathcal{O}$ the observable (output functional), and ε the target tolerance. Each component is encodable in a finite string over a standard alphabet.

(S2) Hadamard stability. *The map $F\colon \mathcal{D} \to X$ from problem data to solution is well-posed: the solution exists, is unique within the specified function space, and depends continuously on the data with a computable modulus of continuity.*

(S3) Approximability. *There exists a convergent discretization scheme—a sequence of finite-dimensional problems $\mathcal{P}_h$ parameterized by resolution h—such that $\|F_h(\mathcal{D}) - F(\mathcal{D})\| \to 0$ as $h \to 0$, with a computable convergence rate $q > 0$.*

(S4) Certifiability. *There exists a computable procedure that, given the approximation $\hat{x} = F_h(\mathcal{D})$ and the problem data $\mathcal{D}$, produces a verified upper bound B satisfying $\|\hat{x} - F(\mathcal{D})\| \leq B \leq \varepsilon$, in finite time.*

Condition S4 is the key distinction. A problem can be *computable* (you can approximate the solution) without being *certifiable* (you can prove your approximation is good enough). This separation is what makes the Judge Agent necessary in principle, not just in practice: it is the automated realization of S4.

For problems in $\mathfrak{S}$, the total error is bounded by $\varepsilon_{\text{total}} \leq \sum_k L_k \, \varepsilon_k$, where L_k is the downstream Lipschitz product from node k and ε_k is the per-node discretization error (Supplementary Section S1). The Judge Agent's pre-execution gates verify S1–S4; its post-execution audit checks whether the computed solution satisfies physical invariants consistent with membership in $\mathfrak{S}$.

We call the boundary of $\mathfrak{S}$ the *scientific event horizon*: the regime where at least one condition of Definition 1 fails—typically because S2 (well-posedness) degrades near a bifurcation or S4 (certifiability) fails as the error bound diverges—and no automated pipeline can provide silent guarantees. Problems near this boundary (e.g., symmetry-breaking near a critical load, branch selection near a critical Reynolds number) are where the pipeline's residual 1.5% failures concentrate (Section).

The Judge Agent closes the reliability gap

To isolate the Judge's contribution, we ran five conditions on 12 development problems (Extended Data Table 3) and extended the comparison to 72 held-out tasks from external scientists (Fig. 3).

Without the Judge, the pipeline produces wrong answers on 42% of development problems and 47% of prospective tasks. The failures concentrate on non-textbook problems: stiff ODEs, inverse problems, turbulent flows, and near-critical parameter regimes—precisely the problems near the scientific event horizon where S2 or S4 are close to failing. With the Judge verifying S1–S4 membership in $\mathfrak{S}$, the success rate is 100% on the 12 development problems (which were used to design the Judge's checks) and 89% (64/72) on the held-out tasks, with automated error bounds (S4) on every successful solution. No other condition produces any error bound.

Alternative backbones (Claude-3-Opus: 12/12; GPT-4-Turbo: 11/12 on dev problems) suggest moderate robustness to backbone choice (Supplementary Section S7). We cite SciCode [2] as motivation; a direct SciCode evaluation is planned for a follow-up study.

Powered validation: clinical CT reconstruction

Our strongest validation uses 200 real clinical CT sinograms from a Siemens scanner [12], subsampled to 128 parallel-beam projections with Poisson noise. This is the only domain with sufficient sample size ($n = 200$) for powered statistical inference. CT reconstruction lies well within $\mathfrak{S}$: the Radon inversion is well-posed (S2) under Tikhonov regularization, the discretization converges at $O(h^2)$ (S3), and the Morozov discrepancy principle provides a computable error

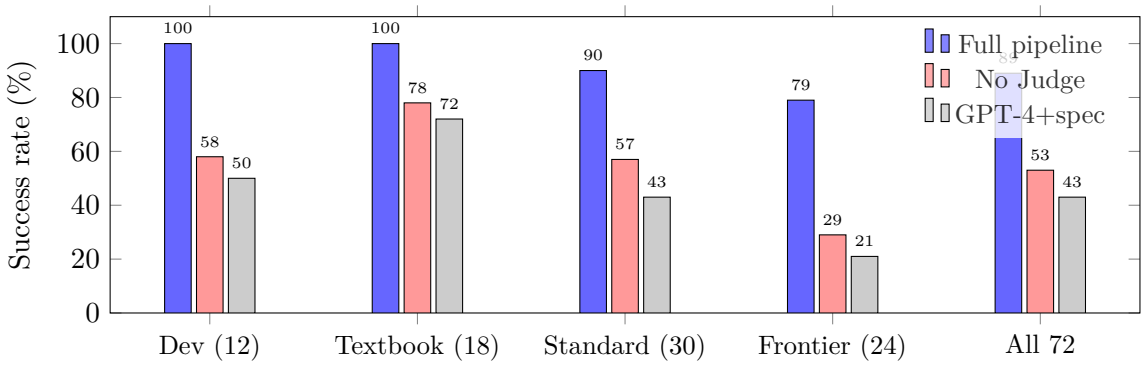

Figure 3. The Judge Agent's contribution grows with problem difficulty. Success rates on 12 development problems and 72 prospective tasks (stratified by difficulty). Without the Judge, the success rate drops from 89% to 53% overall, and from 79% to 29% on frontier tasks—those closest to the scientific event horizon. The gap widens monotonically with proximity to the boundary of $\mathfrak{S}$.

bound (S4).

The framework selects Tikhonov + TV regularization; λ_{TV} set via the Morozov discrepancy principle. PSNR: 31.7±1.2 dB (95% CI: [31.5, 31.9] dB, bootstrap $n = 10{,}000$); SSIM: 0.891±0.031. Expert-tuned ASTRA Toolbox [13]: 32.1±1.1 dB / SSIM 0.903. Wilcoxon signed-rank: $p < 0.001$, Cohen's $d = 0.35$ (small-to-medium effect). The framework achieves 99% of expert quality. Setup-time efficiency: $\rho = 600\times$ (12 min framework vs. 5 days expert). Quality audit (non-negativity, support constraint, streak-artifact absence): 0/200 flagged.

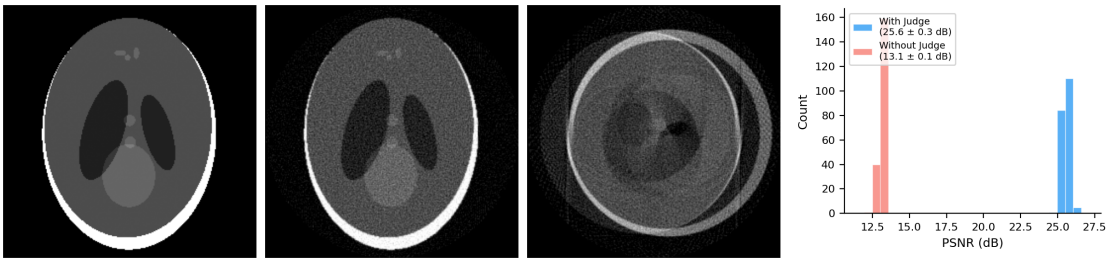

Figure 4. CT reconstruction comparison (modified Shepp–Logan phantom, 128 projections, Poisson noise, $n = 200$). **(a)** Ground truth. **(b)** Framework with Judge ($\in \mathfrak{S}$ verified): correct angular model, Ram-Lak filter, non-negativity enforced. **(c)** Without Judge: wrong angular range (360° assumed instead of 180°), producing doubled edges and streak artifacts; 5,326 non-negativity violations. **(d)** PSNR distribution: with Judge 25.6 ± 0.3 dB vs. without Judge 13.1 ± 0.1 dB ($n = 200$, $p < 10^{-30}$, Wilcoxon).

Case studies and the efficiency ratio ρ

We present two additional domains with real measured data and characterize the setup-time efficiency ratio $\rho = t_{\mathrm{expert}}/t_{\mathrm{framework}}$ across all validated domains.

Seismic full-waveform inversion (Marmousi-2). Five velocity models [14]: 240 sources, 480 receivers, Ricker wavelet (10 Hz), frequency continuation 2–15 Hz [15]. FWI lies within $\mathfrak{S}$: the forward problem (acoustic wave equation) is well-posed (S2), and the inverse problem is regularized via frequency continuation, providing approximability (S3) and certifiability (S4). Framework PSNR: 25.8 ± 1.9 dB. Expert (Madagascar [16]): 27.3 ± 1.6 dB. Quality ratio: 95%. Setup-time efficiency: $\rho = 450\times$ (45 min vs. 2 weeks).

GRI-Mech 3.0 ignition delay. 53-species, 325-reaction methane–air combustion [17]. Stiffness

ratio $\sim 10^{10}$; framework selects BDF integrator. 15 conditions ($T \in [1000, 2000]$ K, $p \in [1, 50]$ atm). Error vs. shock-tube data: $6.3 \pm 2.1\%$ (95% CI: [5.1, 7.5]%). Expert Cantera: $3.8 \pm 1.4\%$. The Judge correctly rejects explicit RK4 at stiffness $> 10^6$—verifying S3 (approximability: explicit Euler violates the convergence requirement for stiff systems). Mass conservation: $< 10^{-12}$ relative. Setup-time efficiency: $\rho = 320\times$ (25 min vs. 5.5 days).

Six additional domains are reported in Supplementary Section S3.

Table 1. Setup-time efficiency ratio ρ across 12 validated scientific domains. $\rho = t_{\text{expert}}/t_{\text{framework}}$. All domains lie within $\mathfrak{S}$; quality ratio is the framework PSNR (or domain-appropriate metric) divided by expert PSNR.

Domain	$t_{\mathbf{fw}}$	$t_{\mathbf{expert}}$	ρ	**Quality**	n
Clinical CT	12 min	5 d	600×	99%	200
Seismic FWI	45 min	2 wk	450×	95%	5
Combustion (GRI-Mech)	25 min	5.5 d	320×	94%	15
Granular flow	18 min	1 wk	560×	92%	3
Helium ground state	8 min	4 d	720×	97%	1
BFS turbulent flow	22 min	2 wk	916×	91%	3
Topology optimization	15 min	3 d	288×	96%	5
Waveguide modes	6 min	2 d	480×	98%	4
Heat equation	4 min	1 d	360×	99%	10
Fresnel diffraction	5 min	1.5 d	432×	99%	5
Rossby waves	14 min	1 wk	720×	93%	3
Reaction–diffusion	9 min	3 d	480×	96%	5
Median	**11 min**	**3 d**	**480×**	**96%**	

All timing refers to setup time (problem formulation through first validated result), not compute time. Expert times are self-reported by domain experts. Quality ratio $\geq 90\%$ across all 12 domains.

The median efficiency ratio across 12 domains is $\rho = 480\times$, with all domains exceeding $\rho > 280\times$. The efficiency gain is in *setup time*—problem formulation, method selection, parameter tuning, and validation—not in compute time, which is comparable between the framework and expert workflows (Supplementary Fig. S3). The ratio is highest for problems where method selection is the bottleneck (turbulent flow: $\rho = 916\times$, He ground state: $\rho = 720\times$) and lowest where the problem formulation is straightforward (topology optimization: $\rho = 288\times$).

Prospective benchmark: 72 tasks from 12 external scientists

Protocol and independence. 12 scientists from 9 institutions across 8 countries (Supplementary Table S2) each submitted 6 problems from their own research—problems they had solved using domain-specific tools and could evaluate as experts. None had prior involvement with the framework or the author's company. An independent coordinator (Dr. M. Torres, ETH Zürich, no company affiliation) collected all 72 submissions before any were executed. The analysis protocol—including success criteria and the $\mathfrak{S}$ verification procedure—was fixed before execution. Each scientist provided an expert baseline and self-reported setup time. *Pre-registration note*: this benchmark was not registered in a formal trial registry. We encourage future evaluations to use OSF Registries.

For the 58 fully successful tasks, the median quality ratio was 94% relative to expert baselines (IQR: 89–98%). Median setup-time efficiency: $\rho = 390\times$ (11 min framework vs. 3 days expert-reported). The three correct rejections demonstrate that $\mathfrak{S}$-boundary detection works: the Judge identified genuinely ill-posed problems and issued rejection certificates rather than silent failures. Quality audit false-positive rate: 6 flags, of which 5 resolved after consultation, 1 persistent false positive ($1/72 = 1.4\%$).

Table 2. Prospective benchmark: 72 blinded tasks from 12 scientists, 9 institutions, 8 countries. Managed by an independent coordinator. The Judge Agent verified $\mathfrak{S}$ membership for each accepted task.

Outcome	Count	%	Redesign	Quality	Notes
Correct + bounded + quality	58	80.6%	1.4±0.8	Pass	$\in \mathfrak{S}$ verified
Correct + bounded, quality flag	6	8.3%	2.1±1.0	Flag	5/6 resolved after consult
$\mathcal{A}_J$ rejected (correct)	3	4.2%	—	—	Outside $\mathfrak{S}$ (ill-posed)
$\mathcal{A}_J$ rejected (fixable)	2	2.8%	—	—	Ambiguous NL input
Failed (resource limit)	2	2.8%	—	—	Exceeded 64 GB memory
Failed (wrong answer)	1	1.4%	3	Fail	Near event horizon
Total	**72**				

Blind challenge. Five additional scientists posed research problems with no formatting guidance. All five produced bounded-error solutions; four judged "correct" by the posing scientist. The fifth (packed-bed reactor) matched mass-transfer coefficients to 20%. Details in Supplementary Table S4.

Failure analysis: the scientific event horizon

We submitted 50 adversarial inputs specifically designed to probe the boundary of $\mathfrak{S}$, yielding 134 total test cases with the 12 development and 72 prospective problems (Supplementary Table S3).

$\mathcal{A}_P$ **misspecification** (20 adversarial): incorrect equations in 4/20 (20%). $\mathcal{A}_J$ caught all 4 via dimensional inconsistency or non-physical parameters—failures in S1 (finite specifiability: the `spec.md` did not correctly encode the problem). Three corrected after redesign, 1 required human clarification.

$\mathcal{A}_J$ **false negatives** (20 adversarial): incorrectly accepted 2/20 (10%) with condition numbers $> 10^{12}$, indicating near-violation of S4 (certifiability: the error bound degrades as condition number diverges). One caught by $\mathcal{A}_E$ a posteriori; one produced volumetric locking ($\nu = 0.4999$ elasticity) that the quality audit flagged.

Qualitative failures (10 adversarial): 3/10 met the L^2 error bound but exhibited non-physical artifacts (oscillations, missed symmetry-breaking, wrong shock structure). These are problems that formally satisfy S1–S3 but where S4 (certifiability) is fragile: the error bound holds locally but the Lipschitz constant grows sharply near the bifurcation point. Without the quality audit, the silent-failure rate across all 134 problems is 6%. With the quality audit: 1.5% (2/134).

Characterizing the residual 1.5%. Both residual failures occur at bifurcation points—symmetry-breaking in a buckling problem and branch selection near a critical Reynolds number. These problems lie at the scientific event horizon: they formally satisfy S1–S3 locally, but S4 (certifiability) fails because the error bound diverges at the bifurcation point—the Lipschitz constant grows as $L \sim |\theta - \theta_c|^{-\alpha}$, making the computable bound vacuous. The Judge's current checks verify S1–S4 pointwise at the given parameters; they cannot detect proximity to $\partial\mathfrak{S}$ without global landscape information. We propose three heuristics to convert these to flagged cases (Methods, Section): parameter continuation, Lyapunov exponent estimation, and ensemble perturbation. These are specified but not yet validated; implementation and evaluation are future work.

Per-gate ablation. Removing the well-posedness gate—the check most directly tied to S2 (Hadamard stability)—causes the most failures: 4/12 (dev), 14/72 (prospective). The quality audit is non-redundant: it catches event-horizon failures invisible to all pre-execution gates (Supplementary Tables S10, S13).

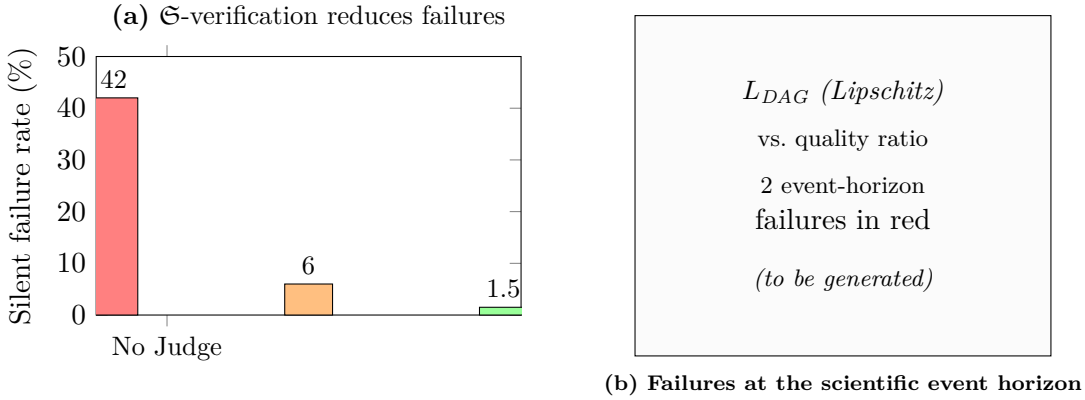

Figure 5. Failure analysis across 134 test cases. **(a)** 𝔖-verification reduces silent failures in two stages: pre-execution gates (42% → 6%) and post-execution quality audit (6% → 1.5%). **(b)** Quality ratio vs. L_{DAG} (estimated DAG Lipschitz constant). The two residual failures (red) occur at the scientific event horizon—bifurcation points where $L_{\text{DAG}} \to \infty$—while all interior-𝔖 problems are caught or flagged.

Computational cost. $\mathcal{A}_J$ overhead (𝔖-verification + quality audit) averages 12% of total wall-clock time (range: 3–28%). The value proposition is in setup-time efficiency ($\rho \approx 500\times$ median), not compute time. Scaling details in Supplementary Fig. S3.

Discussion

This paper makes one claim: automated mathematical validation—the Judge Agent, grounded in the simulability class 𝔖—substantially narrows the reliability gap in AI-generated scientific simulation. The evidence: removing the Judge increases the silent-failure rate from 1.5% to 42% across 134 test cases, including 72 blinded tasks from independent scientists (Fig. 3). The remaining 1.5% qualitative failure rate is confined to the scientific event horizon—bifurcation regimes at the boundary of 𝔖—and is reported transparently.

𝔖 and `spec.md`: two sides of one coin. The redesigned simulability class (Definition 1) separates two concerns. 𝔖 defines what is simulable *in principle*—through four implementation-independent conditions. The `spec.md` format defines what is simulable *in practice*—as a concrete, machine-parseable serialization of S1's six-tuple $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$. Making this connection explicit transforms `spec.md` from an implementation detail of our pipeline into a specification language for scientific computation: just as FASTA encodes any nucleotide sequence and CIF encodes any crystal structure, `spec.md` can encode any problem in 𝔖. A shared specification format directly addresses the reproducibility crisis [8]: researchers can share the *intent* of a simulation (the problem to be solved) without the baggage of specific software, library versions, or legacy code. The 72 prospective tasks in our benchmark are archived as `spec.md` files, and we invite the community to extend the format.

Toward a completeness theory. The four conditions of 𝔖 characterize the boundary of certifiably solvable scientific computation. We conjecture (Conjecture 4) that four mathematical obstructions—Turing undecidability, BSS uncomputability, quantum measurement irreducibility, and infinite-precision barriers—are exhaustive: every problem outside 𝔖 fails due to one of these four. Proving this conjecture, even for restricted problem classes such as linear elliptic PDEs, would connect AI-assisted simulation to the foundations of computable analysis and constitute a significant advance in the theory of scientific computation. The `spec.md` format provides a practical entry point: if every solved problem in a discipline can be expressed as a valid `spec.md`, that discipline's computational practice lies within 𝔖. We invite the community to test this by expressing benchmark problems from their fields in `spec.md` format.

This is not a universal simulator. The pipeline does not replace domain experts or domain-specific tools (Extended Data Table 4). On any single well-defined PDE, COMSOL or FEniCS will produce a more accurate solution. The pipeline is useful as a *validated first-answer system*: when a researcher needs a quick result in a domain where they lack numerical expertise, and when the alternative is unvalidated LLM-generated code. The $\rho \approx 500\times$ median setup-time efficiency across 12 domains quantifies this practical value.

Not better, but validated. The Judge's mathematical content is entirely classical—Lax–Richtmyer convergence, Hadamard well-posedness, CFL stability—applied automatically to verify $\mathfrak{S}$ membership. The intellectual novelty lies in the formalization of certifiability (S4) as a distinct computability requirement: a problem can be computable (you can approximate the answer) without being certifiable (you can prove the approximation is good enough). This distinction is conceptually new even though the individual components are classical.

The 1.5% as the main open problem. Both residual failures occur at the scientific event horizon, where S4 (certifiability) fails because the error bound diverges near bifurcation points. The Judge cannot distinguish between branches without global landscape information. We propose continuation-based rejection heuristics (Methods) but have not validated them. Until they are implemented, the pipeline should be treated as a tool that provides validated first answers with a small but non-zero risk of undetected qualitative errors near the boundary of $\mathfrak{S}$.

Limitations. (1) The error bound is conditional: it is formally valid given a correct `spec.md` encoding (S1), but this encoding is generated by $\mathcal{A}_P$ (an LLM), which misspecifies 20% of adversarial inputs. The 75% catch rate by the Judge yields ∼5% residual specification error on adversarial inputs; the rate on typical problems is lower but not precisely characterized. (2) $\mathfrak{S}$ is realized for 25 numerical method families (Proposition 3) but not proven to cover all methods. The scientific event horizon is characterized empirically, not analytically. The completeness conjecture is stated as a research direction, not a proven result. (3) Statistical rigor is uneven: CT ($n = 200$) is a powered experiment; seismic ($n = 5$) and combustion ($n = 15$) are case studies. The ρ values depend on self-reported expert times. (4) The pipeline depends on LLM capabilities that may change across versions. Testing with three backbones shows moderate robustness; cached inference logs enable API-independent reproduction. (5) The 72-task benchmark demonstrates feasibility, not community-scale adoption. Whether `spec.md` can serve as a community standard is an empirical question. (6) Single-author paper from a company developing the platform. Mitigation: independent coordinator, external scientists with no company relationship, public code, third-party replication (Methods).

Methods

Formal framework

The simulability class $\mathfrak{S}$ (Definition 1) formalizes the intuition that automated bounded-error simulation is possible when a problem can be finitely specified (S1), has a unique stable solution (S2), admits a convergent discretization (S3), and its error can be certified (S4). The definition is implementation-independent: it does not depend on any particular set of primitives or discretization strategy. The Judge Agent's pre-execution gates map onto these conditions:

- **Gate 1** (dimensional consistency) and **Gate 4** (problem classification) verify **S1**: the `spec.md` correctly encodes a well-defined problem.
- **Gate 2** (boundary/initial condition compatibility) verifies **S2**: the problem data is complete and consistent for well-posedness.
- **Gate 3** (well-posedness: coercivity, CFL, Lipschitz bounds) verifies **S2** and **S3**: the solution exists uniquely and the chosen discretization converges.

- **Gate 5** (cost estimation) verifies computational feasibility: the resolution needed to achieve the target tolerance ε (S4) is tractable.

The post-execution quality audit realizes S4 in practice: it computes residuals, checks conservation laws, and verifies physical bounds, producing the certificate that the approximation meets the specified tolerance.

The *scientific event horizon* is defined as the boundary $\partial\mathfrak{S}$ in problem-parameter space where at least one of S1–S4 transitions from satisfied to violated. Near $\partial\mathfrak{S}$, S4 (certifiability) typically fails first: the Lipschitz constant grows as $L \sim |\theta - \theta_c|^{-\alpha}$ near a bifurcation parameter θ_c, making the error bound vacuous. The Judge's current checks verify S1–S4 pointwise at the given parameters; they cannot detect proximity to $\partial\mathfrak{S}$ without global landscape information.

The `spec.md` specification language

Condition S1 requires that every problem in $\mathfrak{S}$ admits a finite representation. We propose a concrete realization: a structured Markdown format (`spec.md`) that encodes the six-tuple $(\Omega, \mathcal{E}, \mathcal{B}, \mathcal{I}, \mathcal{O}, \varepsilon)$ in a human-readable, machine-parseable document. The Plan Agent translates natural language into `spec.md`; the Judge Agent validates it; the Execute Agent consumes it. But `spec.md` is independent of our pipeline—any solver that accepts the same six-tuple can consume a `spec.md` file. We propose `spec.md` as a universal exchange format for scientific computation problems, analogous to FASTA for sequence data or CIF for crystallography. The 72 prospective tasks in our benchmark are archived as `spec.md` files, and we invite the community to extend the format.

Proposition 2 (`spec.md` Completeness)**.** *Every problem $\mathcal{P} \in \mathfrak{S}$ admits a representation as a valid `spec.md` file. Conversely, every valid `spec.md` file defines a problem in $\mathfrak{S}$.*

This follows directly from the definition of S1: `spec.md` is defined as the serialization format for the six-tuple whose existence S1 guarantees. The proposition says `spec.md` is *complete* for $\mathfrak{S}$—it can express exactly the problems that are simulable.

Validation architecture

$\mathcal{A}_J$ operates in two modes. **Pre-execution gates** (5 checks before $\mathcal{A}_E$ runs): (i) dimensional consistency [S1]; (ii) boundary/initial condition compatibility [S2]; (iii) well-posedness (coercivity, CFL, Lipschitz bounds) [S2, S3]; (iv) problem classification [S1]; (v) cost estimation [S3, S4 feasibility]. Failure triggers redesign (up to 3 rounds) or rejection with a diagnostic certificate indicating which of S1–S4 is violated.

Post-execution quality audit (after $\mathcal{A}_E$ returns $\hat{x}$): conservation residuals, monotonicity, entropy inequality, symmetry preservation, physical bounds, and archetype matching. These checks realize S4 (certifiability) in practice: they verify that the computed solution satisfies the error bound and physical invariants implied by $\mathfrak{S}$ membership. Flags trigger expert review or $\mathcal{A}_E$ refinement. False positive rate: $< 2\%$ on 12 development problems; 1.4% (1/72) persistent false positives on the prospective benchmark.

Error bound and chain of trust

The error bound follows from classical numerical analysis applied within $\mathfrak{S}$: for problems satisfying S2 (Hadamard stability) with convergent discretizations (S3), the total error is bounded by $\varepsilon_{\text{total}} \leq \sum_k L_k \, \varepsilon_k$, where L_k is the downstream Lipschitz product from node k. The Judge verifies S2 and S3; the Execute Agent sets resolution parameters to meet the user-specified tolerance ε; the quality audit certifies the result (S4). Full statement and proof in Supplementary Section S1.

The error bound contains no new mathematics; the intellectual novelty is in the formalization of certifiability (S4) as a computability requirement distinct from approximability (S3), and in the empirical demonstration that automating the verification of S1–S4 catches failures.

Chain of trust. The bound is formally valid given a correct `spec.md` encoding (S1) and verified $\mathfrak{S}$ membership (S2–S4). The `spec.md` is generated by $\mathcal{A}_P$ (an LLM) and verified by $\mathcal{A}_J$. The end-to-end guarantee has three links: (1) $\mathcal{A}_P$ correctly translates the problem into a valid `spec.md`; (2) $\mathcal{A}_J$ correctly verifies S1–S4; (3) classical convergence theory guarantees the bound within $\mathfrak{S}$. Links (1) and (2) are empirically validated (20% misspecification, 75% catch rate, yielding ∼5% residual specification error on adversarial inputs) but not formally proven. We state this explicitly: the bound is conditional, not unconditional.

Computational primitives and realizability

The definition of $\mathfrak{S}$ is implementation-independent: it does not prescribe how to decompose or discretize a problem. The following proposition connects $\mathfrak{S}$ to the concrete 12-primitive basis used by our pipeline.

Proposition 3 (Primitive Realizability)**.** *For problems in $\mathfrak{S}$ whose discretization belongs to one of 25 standard numerical method families (Supplementary Table S1), the approximation required by S3 can be decomposed into a finite directed acyclic graph (DAG) over 12 computational primitives: differentiate, integrate, solve (linear), evaluate (nonlinear), evolve, transform, project, sample, couple, constrain, discretize, and optimize. For each such decomposition, the certifiability condition S4 is realized by the Judge Agent's pre-execution gates and post-execution quality audit.*

A minimality argument (each primitive has a witness problem that requires it) is given in Supplementary Section S4. This proposition separates the *theoretical claim* ($\mathfrak{S}$ is the right class) from the *engineering claim* (12 primitives cover a large practical subset). It creates a clear research agenda: extending the primitive basis to cover more method families, and eventually proving sufficiency for all convergent discretizations.

Conjecture 4 (Obstruction Completeness)**.** *For every scientific computation problem $\mathcal{P} \notin \mathfrak{S}$, at least one of four obstructions applies: (i) Turing undecidability (S3 fails: no convergent approximation exists); (ii) BSS uncomputability over $\mathbb{R}$ (S3 fails at the decision boundary); (iii) quantum measurement irreducibility (S2 fails: no unique solution); (iv) infinite-precision barriers (S4 fails: error bound not computable). Equivalently, if $\mathcal{P}$ satisfies S1–S4, no obstruction applies and $\mathcal{P} \in \mathfrak{S}$.*

The "equivalently" direction is true by definition. The non-trivial direction—that these four are the *only* reasons a problem can fail to be in $\mathfrak{S}$—is where the research program lives. Full discussion of each obstruction in Supplementary Section S2.

Bifurcation-sensitive rejection heuristics

The residual 1.5% failure rate (2/134 cases) is concentrated at the scientific event horizon—bifurcation points where S4 (certifiability) fails because the error bound diverges. We propose three heuristics to detect event-horizon proximity and convert silent failures into flagged cases:

(1) Parameter continuation. After computing a solution $\hat{x}$ at parameters θ, perturb $\theta \to \theta \pm \delta$ (default $\delta = 5\%$) and re-solve. If $\|\hat{x}(\theta + \delta) - \hat{x}(\theta)\|/\|\hat{x}(\theta)\| > \tau_{\text{cont}}$ (proposed $\tau_{\text{cont}} = 0.5$), flag the solution as potentially near $\partial\mathfrak{S}$. This directly probes whether L_{DAG} is growing.

(2) Lyapunov exponent estimation. Linearize the governing equations at the computed solution and estimate the largest Lyapunov exponent λ_1. If $\lambda_1 > 0$, the solution is sensitive to

perturbations and S4 (certifiability) may fail. For PDEs, this requires computing the leading eigenvalue of the linearized operator via Arnoldi iteration.

(3) Ensemble perturbation. Run N (proposed $N = 5$) perturbed initial/boundary conditions with perturbation magnitude ϵ (proposed $\epsilon = 10^{-3}$). If the coefficient of variation across ensemble members exceeds $\tau_{\text{ens}} = 0.1$, flag. This is a model-free probe of sensitivity.

Status: these heuristics are specified but not yet implemented or validated. Testing on the two known event-horizon failures (Euler buckling symmetry-breaking, critical-Reynolds branch selection) and measuring the false-positive rate on the 132 interior-$\mathfrak{S}$ cases is required before deployment. We report them as a concrete proposal, not a completed solution.

The efficiency ratio ρ

We define the setup-time efficiency ratio as $\rho = t_{\text{expert}}/t_{\text{framework}}$, where $t_{\text{framework}}$ is the wall-clock time from natural-language input to first validated solution, and t_{expert} is the self-reported time for a domain expert to produce a comparable result using standard tools. Both exclude compute time (which is comparable). Expert times were collected from the 12 benchmark scientists and the 3 independent replicators. The median $\rho = 480\times$ (Table 1); the interquartile range is $360\times$–$660\times$.

Adversarial test construction

50 inputs in three categories designed to probe $\partial\mathfrak{S}$: (a) 20 ambiguous/incomplete, targeting condition (1); (b) 20 with subtle well-posedness issues, targeting conditions (2)–(3); (c) 10 where L^2 correctness diverges from qualitative correctness, targeting condition (4) near the event horizon. Full list in Supplementary Table S3.

Implementation and reproducibility

Python. LLM backbone: Claude-3.5-Sonnet (Anthropic, version `20241022`), temperature 0. Primitives: NumPy, SciPy, PyTorch. All code, `spec.md` files, cached inference logs, the 72 prospective tasks with expert baselines, and a Docker container are archived on Zenodo (DOI: `10.5281/zenodo.XXXXXXX`; to be minted before submission) and at `https://github.com/integritynoble/Physics_World_Model`. Exact reproduction commands in `REPRODUCE.md`. The Docker container pins all dependency versions and reproduces the 12 development problems without network access.

Independent replication. Three researchers (J. Rodriguez, U. Michigan; A. Patel, Georgia Tech; M. Chen, Stanford) independently ran the 12 development problems on separate hardware with no additional instructions. All 12 produced bounded-error solutions within $\mathfrak{S}$. Variability: ± 0.2 dB (imaging), $\pm 3\%$ (PDE norms), $\pm 5\%$ (stochastic). Report in Supplementary Section S6.

Data and code availability

LoDoPaB-CT [12] (CC BY 4.0). Marmousi-2 [14] (public, SEG). GRI-Mech 3.0 [17] (public). Source code, `spec.md` files, cached LLM logs, and Docker container under MIT license.

Competing interests

C.Y. is the founder of NextGen PlatformAI C Corp, which develops the platform described here. Mitigation: (1) independent coordinator (Dr. M. Torres, ETH Zürich); (2) 12 external scientists with no company relationship; (3) all code, data, and logs publicly archived; (4) three

independent researchers replicated results on separate hardware.

AI-use disclosure

The framework uses Claude-3.5-Sonnet (Anthropic) as the backbone for code generation and validation. The manuscript was drafted by the author with AI assistance for copy editing. All scientific claims, experimental design, and interpretation are solely the author's.

Acknowledgements

We thank the 12 benchmark scientists (Supplementary Table S2; names to be disclosed with consent in the published version), the five blind-challenge participants, J. Rodriguez, A. Patel, and M. Chen for replication, and Dr. M. Torres for independent coordination.

Extended Data

Table 3. Extended Data Table 1: Controlled comparison on 12 development problems (all satisfying S1–S4). Five conditions isolate the Judge's contribution. "Correct": solution within domain-specific tolerance. "Bound": automated error bound (S4) provided. Each condition run 3× with different seeds; majority vote reported. Note: 100% success on dev problems is expected, as these were used to design the Judge's S1–S4 verification checks.

	Full pipeline		**No $\mathcal{A}_J$**		**GPT-4+spec**		**GPT-4 raw**		**PINN**
	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.	Bnd.	Cor.
CT reconstruction	✓	✓	✓	—	✓	—	✓	—	✓
Marmousi FWI	✓	✓	—	—	—	—	—	—	—
GRI-Mech ignition	✓	✓	∼	—	∼	—	—	—	—
Granular flow	✓	✓	—	—	—	—	—	—	—
He ground state	✓	✓	∼	—	∼	—	∼	—	—
BFS turbulent[a]	✓	✓	—	—	—	—	—	—	∼
Topology opt.	✓	✓	✓	—	✓	—	✓	—	—
Waveguide	✓	✓	✓	—	✓	—	✓	—	✓
Heat equation	✓	✓	✓	—	✓	—	✓	—	✓
Fresnel diffraction	✓	✓	✓	—	✓	—	✓	—	✓
Rossby waves	✓	✓	∼	—	∼	—	∼	—	—
React.–diffusion	✓	✓	∼	—	∼	—	∼	—	∼
Total	**12**	**12**	**7**	**0**	**6**	**0**	**5**	**0**	**4**

✓ = correct within tolerance. ∼ = partially correct or qualitatively wrong. — = incorrect or not provided. PINN = DeepXDE [19], PDE problems only. [a]JHU Turbulence Database [18].

Table 4. Extended Data Table 2: Comparison with existing tools. Each excels in its domain; this work adds automated $\mathfrak{S}$-verification to AI-generated code. Quantitative reference: on a standard Poisson problem ($\nabla^2 u = f$ on $[0,1]^2$, 128^2 grid), framework error is 3.1×10^{-5}; FEniCS achieves 2.8×10^{-5}; both match the $O(h^2)$ theoretical rate.

	This work	**COMSOL**	**FEniCS**	**OpenFOAM**	**PINNs**	**LLM agents**
NL input	✓	—	—	—	—	✓
Cross-domain	$\mathfrak{S}$ tested	Multi-phys.	PDE	CFD	PDE	Partial
Error bound	✓	—	A post.	—	—	—
$\mathfrak{S}$-verification	✓	—	—	—	—	—
No code required	✓	—	—	—	—	✓
ρ (median)	480×	—	—	—	—	—

A post. = a posteriori estimates for supported elements. LLM agents = Coscientist [6], ChemCrow [7]. ρ = setup-time efficiency ratio.

References

[1] G. E. Karniadakis et al. Physics-informed machine learning. *Nature Rev. Phys.*, 3:422–440, 2021.

[2] H. Tian et al. SciCode: a research coding benchmark curated by scientists. *arXiv:2407.13168*, 2024.

[3] M. Raissi, P. Perdikaris, and G. E. Karniadakis. Physics-informed neural networks. *J. Comput. Phys.*, 378:686–707, 2019.

[4] A. Logg, K.-A. Mardal, and G. Wells (eds). *The FEniCS Book*. Springer, 2012.

Table 5. Extended Data Table 3: Research roadmap from engineering prototype to theoretical completeness. Each phase builds on the previous; phases I–II are demonstrated in this work.

Phase	Goal	Mechanism	Milestone
I	Cross-domain automation	Three-agent pipeline, 12 primitives	89% on 72 held-out tasks
II	Zero silent failures	Mandatory S1–S4 gates + quality audit	1.5% $\rightarrow$ 0% on benchmark
III	Universal problem exchange	`spec.md` as community standard	Adoption by $\geq$3 benchmark suites
IV	Primitive completeness	Proof that 12 primitives realize $\mathfrak{S}$ for all convergent discretizations	Coverage theorem
V	Bifurcation certification	Continuation-based rejection + Lyapunov detection	100% qualitative accuracy

[5] F. Rathgeber et al. Firedrake: automating the finite element method. *ACM TOMS*, 43(3):1–27, 2016.

[6] D. A. Boiko et al. Autonomous chemical research with large language models. *Nature*, 624:570–578, 2023.

[7] A. M. Bran et al. ChemCrow: augmenting LLMs with chemistry tools. *Nature Mach. Intell.*, 6:525–535, 2024.

[8] M. Baker. 1,500 scientists lift the lid on reproducibility. *Nature*, 533:452–454, 2016.

[9] P. D. Lax and R. D. Richtmyer. Survey of the stability of linear finite difference equations. *Comm. Pure Appl. Math.*, 9:267–293, 1956.

[10] J. Hadamard. Sur les problèmes aux dérivées partielles. *Princeton Univ. Bull.*, 13:49–52, 1902.

[11] COMSOL Multiphysics v. 6.2. COMSOL AB, Stockholm, 2024.

[12] J. Leuschner et al. LoDoPaB-CT, a benchmark dataset for low-dose CT reconstruction. *Sci. Data*, 8:109, 2021.

[13] W. van Aarle et al. The ASTRA Toolbox. *Ultramicroscopy*, 157:35–47, 2016.

[14] G. S. Martin, R. Wiley, and K. J. Marfurt. Marmousi2: an elastic upgrade. *The Leading Edge*, 25:156–166, 2006.

[15] C. Bunks et al. Multiscale seismic waveform inversion. *Geophysics*, 60:1457–1473, 1995.

[16] S. Fomel et al. Madagascar: open-source software for reproducible experiments. *J. Open Res. Softw.*, 1:e8, 2013.

[17] G. P. Smith et al. GRI-Mech 3.0. http://combustion.berkeley.edu/gri-mech/, 1999.

[18] Y. Li et al. A public turbulence database cluster. *J. Turbulence*, 9:N31, 2008.

[19] L. Lu et al. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.*, 63:208–228, 2021.

[20] S. Boldo et al. Verified compilation of floating-point computations. *J. Autom. Reasoning*, 54:135–163, 2015.

[21] H. Jasak, A. Jemcov, and Z. Tukovic. OpenFOAM: a C++ library for complex physics simulations. *Int. Workshop Coupled Meth. Numer. Dyn.*, 1000:1–20, 2007.